databricks

eBook

Operationalizing Agent Fleets at Scale

Contents

Introduction: Why Enterprise AI Breaks at Scale 3

The last-mile problem of operationalizing agents 4

From one-off agents to a shared build system 5

Agent quality depends on business-aware context 6

Governance as the control plane for agent fleets 7

The Agentic Platform: Choice, Context and Control 8

Choice: Build and run agents without rebuilding the stack 9

Context: Ground agents in governed data and business semantics 11

Control: Centrally govern and optimize agents in production 12

Agents at Work: How Enterprises Put AI to Use 13

Multi-agent workflows in the enterprise 13

Best Practices for Evaluating Agent Platforms 16

Summary 17

About Databricks 18

Introduction: Why Enterprise AI Breaks at Scale

Ask 10 enterprise teams what an agent is and you'll get 10 different answers. For some, it's a coding assistant. For others, it's a system that retrieves data, invokes tools and takes action inside a business workflow. The label is shared, but the scope isn't. And the wider that scope becomes, the more quickly enterprises run into the real work of adoption.

The challenge is no longer proving that an agent can be built. It's deciding how fleets of agents should participate in the work of the business and then building the system that lets them run safely and reliably at scale. Again and again, the same patterns show up: agents work in a pilot but struggle in production; they look smart in isolation but break on real data and documents; they're built differently by every team; and as fleets grow, governance falls behind.

For decision-makers, there are four key considerations:

The last-mile problem of operationalizing agents: When agents move from pilots into real workflows, the operating layer may not exist, forcing teams to build scaffolding.

From one-off agents to a shared build system: Without a shared meta layer, as multiple teams build agents, every group recreates identity, data access, orchestration and evaluation.

Agent quality depends on business-aware context: Agents fail not because models cannot reason, but because business context is fragmented.

Governance as the control plane for agent fleets: Managing reliability, security and cost requires unified enterprise oversight across data, models, tools and runtime policies.

These challenges exist across organizations regardless of size or industry. For example, imagine a large insurer that launches an internal claims assistant.

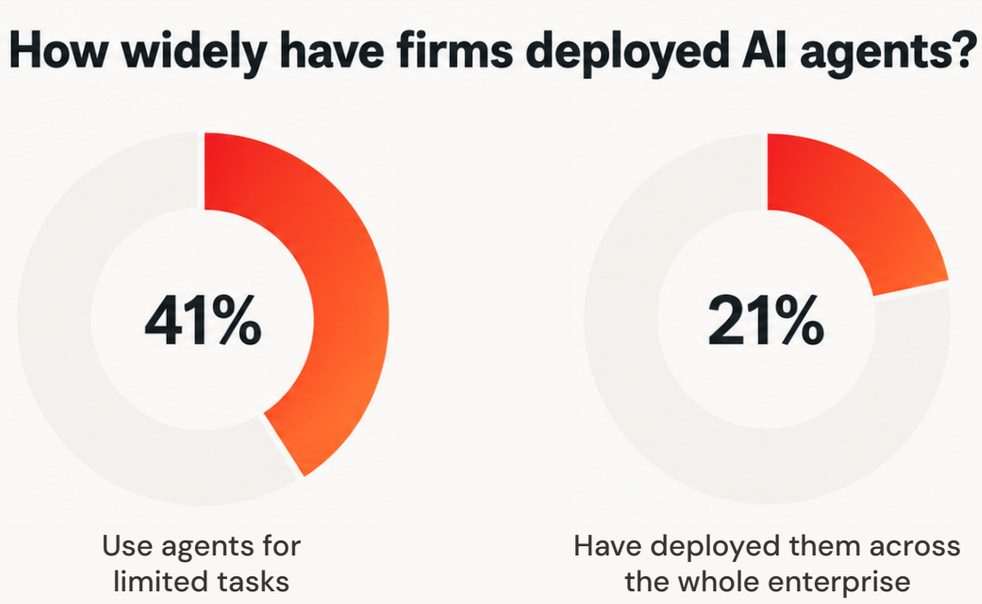
It answers adjuster questions, pulls policy details and summarizes documents fast enough to impress executives. But as the company tries to expand that assistant across claims, customer service, fraud review and compliance, new problems appear. The assistant behaves differently depending on which workflow it's in, different teams rebuild the same plumbing, document retrieval breaks on edge cases, and no one has a clear view into what the system does or how much it costs. That's the moment when "we built an agent" gives way to the much harder question: **can we actually run agents as an enterprise system?**

The last-mile problem of operationalizing agents

When agents move from a pilot into real business workflows, the last-mile gap shows up. The agent itself may be fine, but the operating layer around it is missing. Teams suddenly need a stable runtime to deploy and isolate agents and their tools, secure integrations into existing systems, observability to see what the agent did and why, runtime guardrails to control what it's allowed to do, and cost management to track and attribute spend. Instead of improving the agent's behavior, they're hand-building the scaffolding for every project.

In the insurer example, the first version of the claims assistant worked while it was closely supervised in one workflow. As soon as it was expected to operate across multiple lines of business, the team needed more than a smarter agent. They needed a way to run it repeatedly and safely, keep its actions auditable, enforce which data and tools it could touch and change models or configurations without rewriting the system.

That's the shift leaders feel most quickly: the question stops being whether an agent can answer a question and becomes whether the organization has the runtime, guardrails and visibility to let that agent participate in real work at scale.



Economist Enterprise survey of 1,200+ technology executives, *Making AI Deliver*, 2026.

Figure 1. State of AI agent adoption.

From one-off agents to a shared build system

Once multiple teams start building agents, the real friction is not the models; it's the way agents are built and wired together. At pilot stage, each group can pick a favorite agent harness or coding assistant such as Claude Code or Cursor to ship an agent, but that model breaks down as soon as you have several agents in production.

Without a shared way of building, each group recreates the same work: one team integrates identity and data access into its chosen harness, another reimplements authentication and orchestration logic and a third bakes workflow patterns and evaluation into its own framework.

The insurer's claims, fraud and service teams may each build their own agent with separate tool wrappers, retrieval patterns and evaluation methods, so on paper the company has a fleet of agents, but in practice it has isolated projects that happen to call models and can't be composed into something larger.

This mirrors how engineering teams already work with AI tools more broadly. Recent GitLab research found that 91% of organizations have two or more AI coding tools in active use and 54% have three or more.

As developers increasingly use different tools for different stages and types of work, multi-tool, multi-harness environments have become the default rather than the exception.

The problem is structural: each harness understands only its own sessions. As soon as you want to combine agents, govern them or let multiple people work with them together, you hit a ceiling. Engineers start designing loops and multi-agent workflows, but there's no common layer to orchestrate teams of agents, enforce shared policies like cost budgets and permissions or give everyone a consistent interface for messages, files and tool calls. What looks like healthy experimentation at first becomes fragmentation: more agents, but no system for creating, combining and controlling them.

Enterprises that move past this stage stop treating each agent as its own mini platform and start investing in a shared build system — a meta layer above individual harnesses. In that layer, tools are reusable across agents and harnesses, models are interchangeable behind a common interface, and policies and collaboration live in one place instead of inside prompts or ad hoc scripts. The question shifts from "How do we ship this one agent?" to "How do we give every team a consistent way to build, compose and govern agents as part of a larger system?"



Agent quality depends on business-aware context

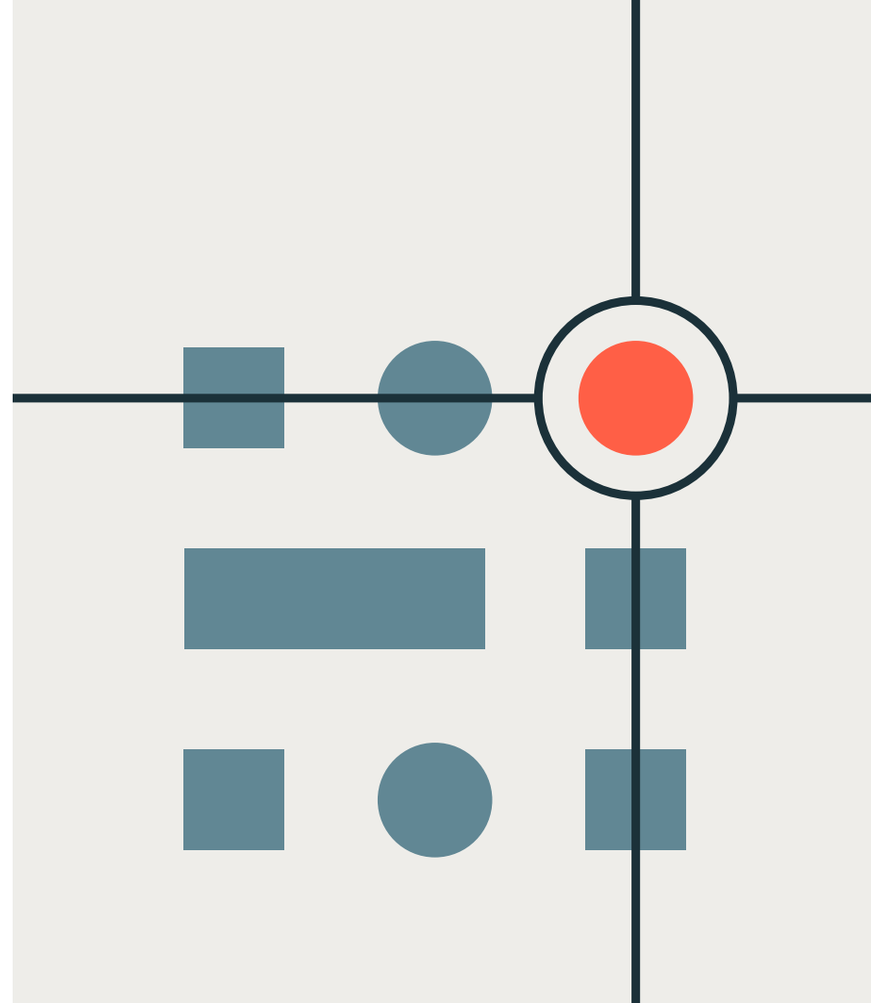
Once enterprises begin operationalizing agents, the next constraint is context. Agent quality depends less on raw model power than on whether the agent actually understands the business: the data, documents, entities, relationships and history it's working over.

This is where many promising agents begin to fail. They miss the key clause in a contract, join the wrong tables or choose the wrong tool, not because the model can't reason, but because the business context is fragmented.

According to Gartner, organizations deploy an average of 897 applications, scattering data, definitions and institutional knowledge across hundreds of unintegrated silos. Semantics are hard-coded per application, retrieval pipelines are rebuilt for each use case and memory is scattered across different systems. In the claims example, one workflow draws from policy PDFs, another from CRM records and another from adjuster notes in a separate system, so the same assistant gives different answers depending on where it's running.

Enterprises that move past this stop treating context as bespoke per-agent logic and start treating it as a shared layer. They invest in a consistent ontology for data and documents, shared business semantics and unified memory so agents can reason over the same source of truth across channels and workflows.

When context becomes a durable asset instead of hidden plumbing, the question shifts from "How do we make this one agent smarter?" to "How do we ensure every agent operates on the same business understanding?"



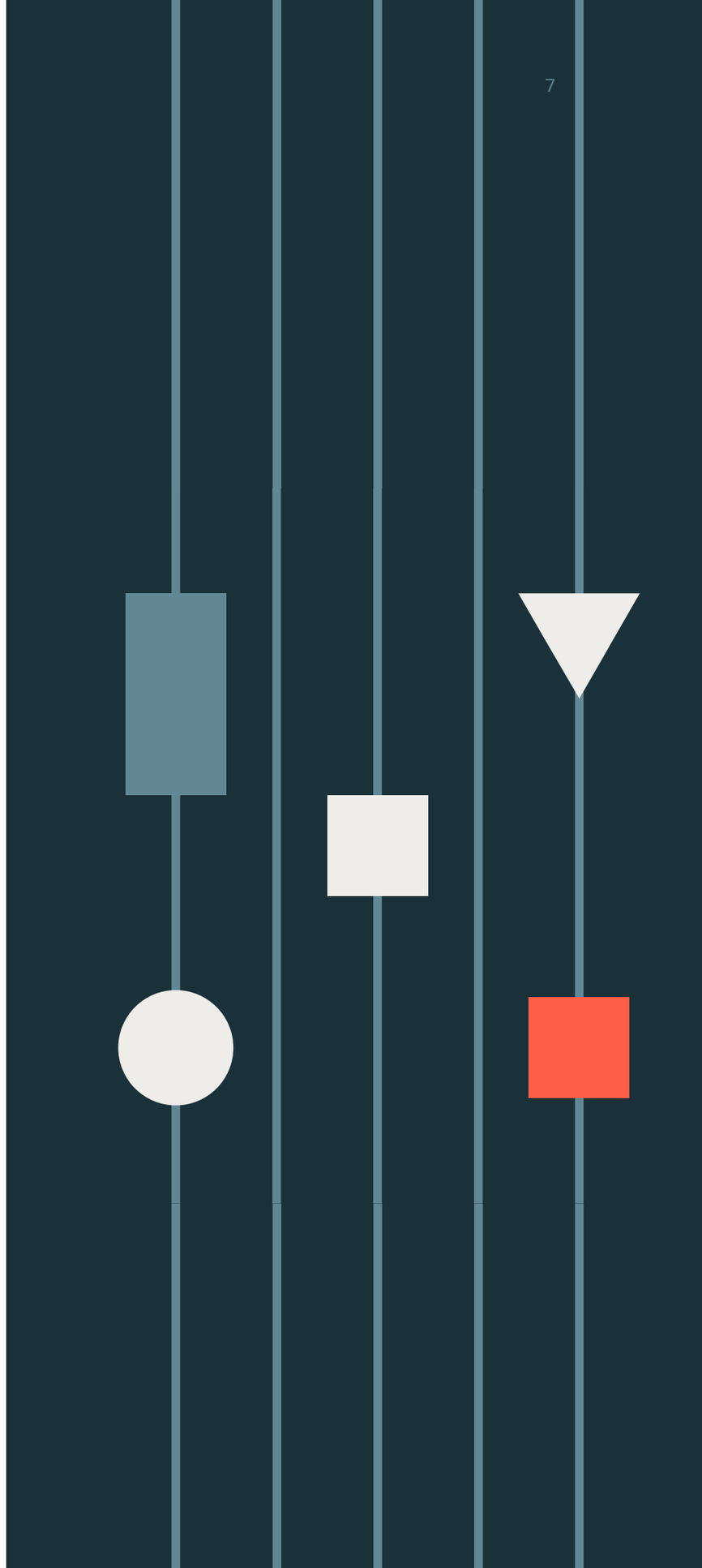
Governance as the control plane for agent fleets

As agents move deeper into business processes, governance is no longer limited to who can access a model or table. It needs to extend to what agents can access, what tools they can invoke, what actions they can take, how they're monitored and how their costs are controlled. This is especially important since most organizations don't start from a clean slate. They govern data, models, agents and enterprise tools through separate systems, which creates fragmented visibility, inconsistent policies and duplicated governance work.

Runtime AI governance is often even weaker: static permissions, isolated guardrails and disconnected monitoring tools provide limited control over agent behavior and rapidly growing AI spend. In insurance claims, that means one team can see model logs, another can see data access records and a third can estimate cost, but no one can answer the full question of what happened when an agent took an action, what context it used and whether that behavior was policy-compliant.

According to Gartner, by 2027, 40% of enterprises will demote or decommission autonomous AI agents because governance gaps are only discovered after incidents in production.

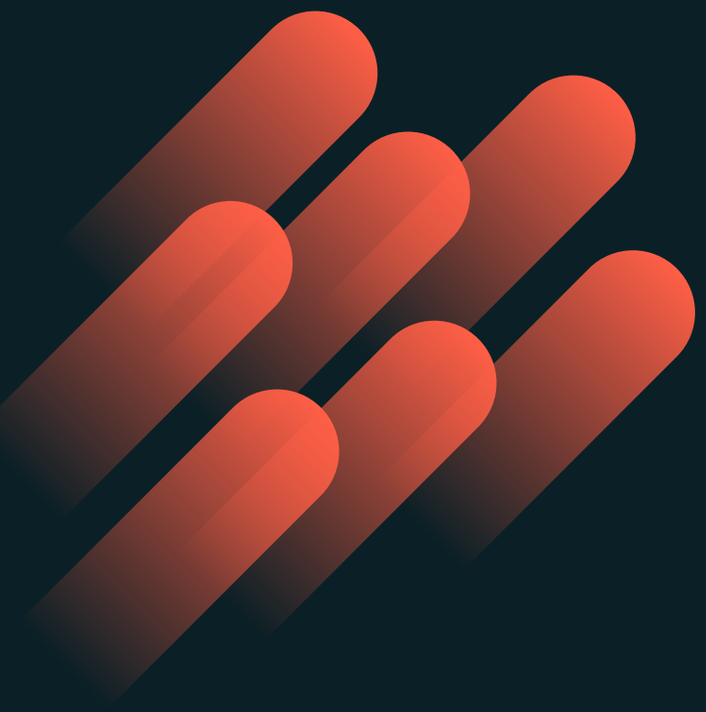
That's why governance is becoming the control plane for enterprise AI. At fleet scale, organizations can't manage reliability, compliance and cost as separate processes. They need a unified way to govern data, models, agents, tools, policies and runtime interactions together. Without that operating layer, every new agent increases uncertainty. With it, every new agent becomes easier to understand, control and improve.



The Agentic Platform: Choice, Context and Control

Enterprise teams that solve the four challenges do so by treating agents as part of a unified platform, not as one-off projects. The practical question for decision-makers becomes: what should that platform actually do and how should it be structured, so agents can be built, run and governed consistently on top of business data and tools?

Databricks approaches this with Agent Bricks agent platform for three linked priorities: choice in how agents are built, context in how they understand the business and control in how they are governed in production. Each priority maps directly back to the last-mile, shared-build-system, context and governance gaps described earlier.



Choice: Build and run agents without rebuilding the stack

Agent Bricks lets teams build agents with the tools they already use, including coding assistants, frameworks, SDKs and MCP servers.

Internal Databricks research shows that developers already use three to five coding agent tools simultaneously, underscoring the need to support a range of tools without rebuilding the underlying stack.

A shared runtime layer handles deployment, isolation and workflow execution, while governed model endpoints expose frontier and open-source models with routing, fallbacks and budgets.

Agent Bricks is designed around that principle, so organizations spend less time rebuilding runtimes and more time improving agents and can change underlying AI components as the market moves, instead of being locked into early choices.

For leaders, the implication is straightforward: avoid architectures where every new agent requires a new stack. The target state is an environment where teams can swap models, harnesses or vendors based on evidence and cost–quality tradeoffs, without touching downstream workflows.



Figure 2. The lifecycle of agent development.

THE META-HARNESS DEMYSTIFIED

If you're new to agents, it helps to distinguish them from the harnesses they run on. An agent is a process defined around a specific task and a set of tools: for example, reviewing claims, handling support tickets or editing code, with access to search, databases and execution. A harness is the runtime that wires those pieces together — which models are called, how tools are invoked, how state is passed between steps and how traces and errors are captured.

A meta-harness sits above those individual runtimes. It gives you a common way to register different harnesses, compose agents that use more than one and apply consistent policies and observability across them. The practical benefit is that teams don't have to standardize on a single framework or rebuild orchestration every time they want to try a new coding agent or tool stack; they can mix and match while keeping a stable operational surface.

Omnigent is an open-source meta-harness Databricks built for this purpose. It lets teams plug in agents built on various SDKs or MCP servers, combine them into shared workflows and manage them through one layer of evaluation, tracing and control. That makes experimentation safer because new harnesses still inherit the same guardrails and it makes reuse easier because successful patterns and tools can be shared across agents instead of being trapped inside one framework.

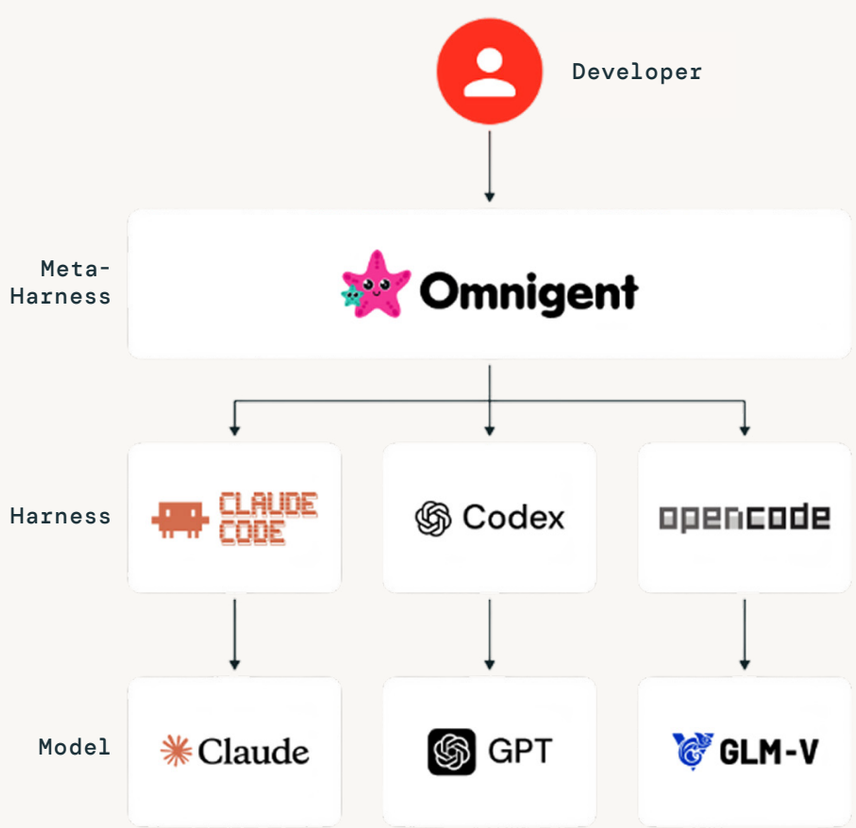


Figure 3. Omnigent is the open-source meta-harness for AI agents.

Context: Ground agents in governed data and business semantics

Solving the context problem means giving agents a consistent, governed view of the business across data and documents. In a unified platform, agents connect to the lakehouse as their main source of structured data and documents and to third-party apps through governed connections. On top of this, a shared context layer encodes business semantics via ontology, while search, document understanding and memory services help agents find the right information and remember past interactions.

In practice, that means each new agent should start from the same trusted business view instead of its own custom pipelines. Answer quality improves because agents see the same entities, relationships and history wherever they're used, and teams spend less time wiring RAG and semantics per use case and more time designing workflows that matter.

With Agent Bricks, this context is embedded directly into retrieval and planning, delivering 70% higher accuracy than standard RAG and a 30% improvement in multi-step workflows.

In internal Databricks benchmarks, an agent grounded in this ontology answered 84.5% of questions correctly on the first attempt, while the strongest general-purpose coding agent managed 52.4% and the weakest only 25%, and the ontology-backed agent did so at roughly 2x lower latency than the strongest baseline.

For decision-makers, the lesson is to treat business context as a platform capability: something every agent inherits, not something each team rebuilds.

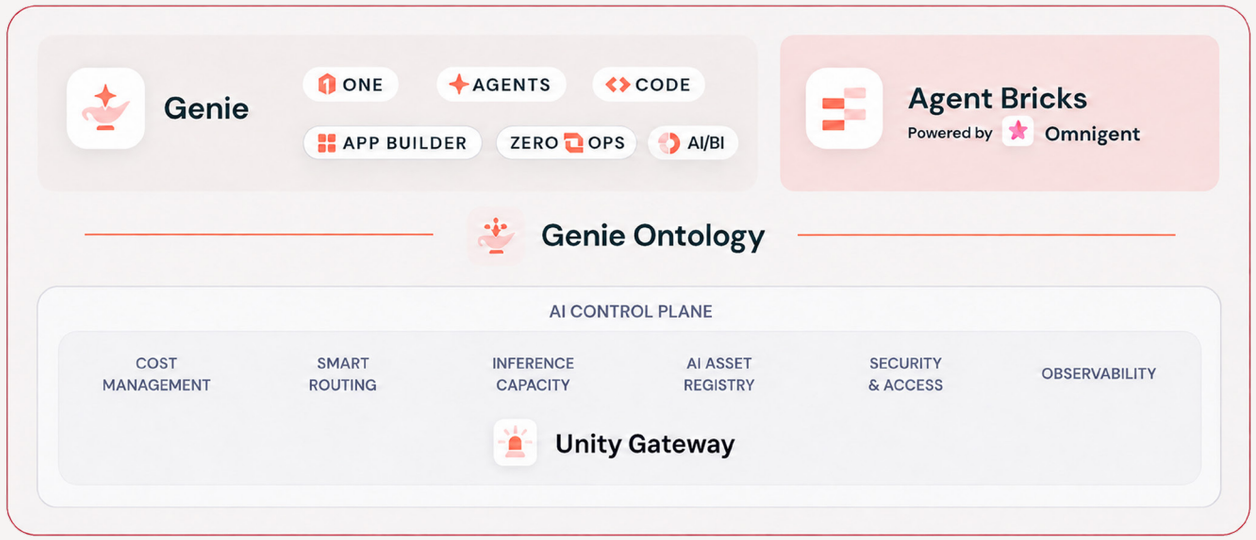


Figure 4. The Databricks Platform Agentic Dev & Work and AI Control Plane

Control: Centrally govern and optimize agents in production

Finally, a platform for agent fleets needs to act as a control plane. Agents, models, tools and external AI services should be registered with shared permissions, policies, lineage and audits. When agents run, they should do so in secure, isolated environments with down-scoped data access, and their activity should be captured with standard traces, evaluations, telemetry and budgets.

With that layer in place, there is a single place to see what agents are doing and to adjust behavior over time. Reliability, security and cost are managed as part of one control plane, so adding agents increases visibility rather than uncertainty, and evaluation data can be reused to tune prompts, tools and model choices across the portfolio instead of team by team.

Databricks' Agent Bricks, combined with Unity Gateway, is structured to provide this kind of unified view, but the underlying pattern is general: consolidate policies and monitoring where agents actually execute, rather than around individual models or applications.

Unity Gateway provides the runtime control layer for these interactions. It provides organizations with centralized visibility into how agents use models and tools, along with guardrails and contextual policies to protect sensitive data and control agent actions. It also brings usage and cost into the same control plane, with granular attribution, budgets, hard spend caps and smart routing to help teams manage and optimize AI spend as agent fleets scale.

For leaders, the prescription is clear: treat governance as the operational backbone of agent fleets. Choose or build a platform that offers both the flexibility to mix tools and harnesses and the discipline of shared context and control, so that every new agent makes the overall system easier to reason about, not harder.



Agents at Work: How Enterprises Put AI to Use

Multi-agent workflows in the enterprise

Teams often begin with multi-agent workflows in domains where work is high-stakes, spans multiple systems and document types and already follows well-understood processes that agents can be trained and governed against.

Investment and research analysis: Asset management, banking and strategy teams have repeatable processes for reading filings and research, joining them with internal and market data, and producing structured comparisons or recommendations. Multi-agent setups can split this work across document analysis, data retrieval and synthesis agents, while keeping decisions auditable.

Regulatory and legal review: Utilities, insurers, financial institutions and legal teams manage large bodies of filings, contracts and policies, with clear review patterns and strict compliance requirements. Coordinated agents can handle search, extraction and drafting, while human experts stay in the loop for judgment and sign-off.

Customer and operations workflows: Claims, service and support organizations run high-volume, rule-driven processes that already span CRMs, policy systems and knowledge bases. Multi-agent workflows can route tasks, retrieve context, summarize history and propose next actions, all under shared guardrails and metrics.

Healthcare and other safety-critical assistants: Healthcare providers and life sciences teams need assistants that combine triage, education and escalation against approved content and clinical logic. Different agents can focus on symptom intake, content retrieval and safety checks, while governance ensures consistent behavior across scenarios.

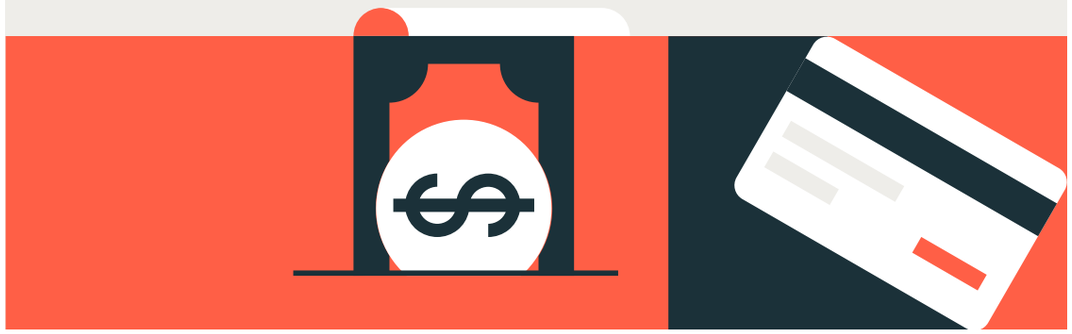
Enterprise data analysis and decision support: Finance, operations and analytics teams rely on recurring analyses that cross data warehouses, dashboards and business documents. Multi-agent patterns let one agent plan the analysis, another run queries and checks and a third produce narratives or decks, all grounded in the same governed data model.

CASE STUDY 01/03

Scaling investment analysis at Franklin Templeton

Franklin Templeton uses Agent Bricks to turn what used to be days of manual fund analysis into seconds of agent-driven insight. Their use case centers on an "analyst" agent that reads fund documents, internal research and market data, then produces structured commentary and comparisons for investment teams. Agent Bricks provides the agent orchestration and evaluation surface, Omnigent lets them mix different models and tools as they refine workflows and Unity Catalog keeps the underlying data and models governed.

The benefit is twofold: analysts save hours per week on repetitive reading and summarization and the organization gets more consistent, repeatable analysis instead of one-off notes. That, in turn, makes it easier to spot opportunities across a broader universe of funds because the agent is doing the heavy lifting over a shared, trusted context.



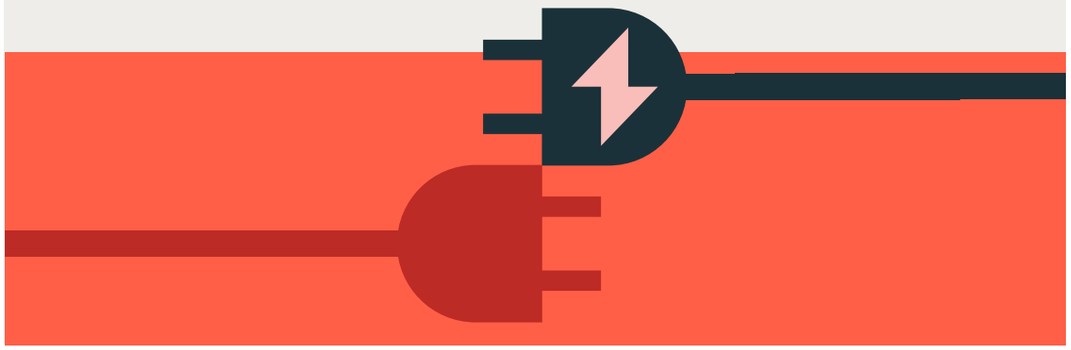
CASE STUDY 02/03

Operationalizing regulatory search at Hawaiian Electric

Hawaiian Electric uses Agent Bricks to power a regulatory assistant that helps teams search and interpret more than 60,000 regulatory filings. The agent's job is simple to describe but hard to execute: find the right filings, extract relevant passages and present grounded answers that legal and regulatory staff can trust.

Here, Agent Bricks provides the agent layer, Unity Catalog governs which documents and tools the agent can access and Genie Ontology plus Document Intelligence helps the agent understand and navigate complex regulatory text.

The outcome is faster, more reliable regulatory research. Staff can find and reuse the right language quickly, while maintaining confidence that the agent operates within the correct document set and governance policies. What started as a search problem becomes a governed agent workflow that shortens review cycles and reduces manual effort.



CASE STUDY 03/03

Delivering safer health insights at Flo Health

Flo Health uses Databricks to support a health assistant that answers sensitive questions about topics like menstrual cycles, symptoms and lifestyle. Their agent must balance helpfulness with medical safety, grounding answers in trusted sources and carefully tuned behavior.

Agent Bricks gives them a platform to define and evaluate the assistant's workflows. Lakehouse and Unity Catalog unify clinical content, behavioral data and app telemetry under one governance model. Agent Bricks' evaluation and tracing tools help Flo test agents against medical safety criteria, refine prompts and tool usage and compare different model setups.

The benefit is a measurable improvement in medically acceptable responses, at lower cost than bespoke model development. Flo can extend the assistant to new areas (like sleep or fertility) with confidence that the underlying system will enforce the same safety and governance standards.



databricks

"The ability to run experiments at this scale has fundamentally changed our approach to product development."

— Dmitry Zolotukhin, VP of Analytics at Flo Health

Best Practices for Evaluating Agent Platforms

As organizations move from prototypes to production, their requirements mulitply. The real question is not whether a platform can help build an agent, but whether it can support agents as durable parts of the business.

Three core actions to prioritize:

1

Choose a platform that absorbs the operational heavy lift

Start with the realities of production: deployment, isolation, tool safety, tracing, evaluation and cost controls. If teams still have to assemble that operating layer for every new agent, the platform is not doing enough.

2

Treat context as a core capability

Look for a unified context layer that combines structured and unstructured data from documents, external systems and business semantics. If every agent needs its own retrieval pipeline and hand-coded view of the business, quality will drift and scaling will get expensive.

3

Make unified governance non-negotiable

Insist on a single view of which agents exist, what they are doing, what they cost and how they are governed. If answering those questions means stitching together logs, policy engines and admin tools, governance is not truly unified.

Summary

As “agent” shifts to mean systems that retrieve, reason and act in real workflows, enterprises are moving from proving what one agent can do to deciding how agents fit into the way the business runs. The most durable architectures treat agents as part of an operating layer, with a control plane for data, context, execution and governance that extends strategies customers already have.

When organizations pair that platform approach with their own domain expertise and processes, agents become another way the business operates day to day — visible, accountable and tied directly to outcomes that matter.

Explore Agent Bricks

databricks

Get Started with the Databricks Platform

Start your free trial

Level up your AI agent skills

AI Agent Fundamentals

Recommended resources

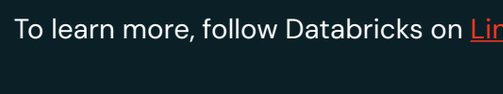
A Compact Guide to AI Agents

2026 Gartner® Magic Quadrant™ AI Platforms for DSML

About Databricks

Databricks is the data and AI company. More than 10,000 organizations worldwide — including Block, Comcast, Condé Nast, Rivian, Shell and over 60% of the Fortune 500 — rely on the Databricks Data Intelligence Platform to take control of their data and put it to work with AI. Databricks is headquartered in San Francisco, with offices around the globe and was founded by the original creators of Lakehouse, Apache Spark™, Delta Lake and MLflow.

To learn more, follow Databricks on LinkedIn, X and Facebook.



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark and the Spark logo are trademarks of the Apache Software Foundation. Privacy Policy | Terms of Use